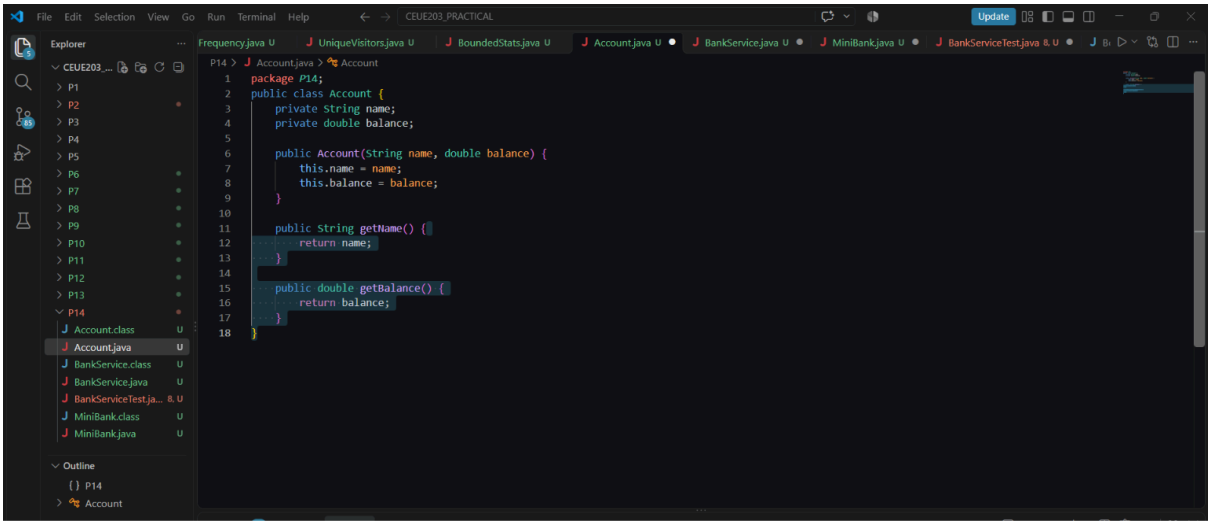
# PRACTICAL 14

QUESTION

1. Assemble and test: build a tiny layered app (a model, a service using an ArrayList, and a menu main), then write two or three JUnit @Test methods checking that adding increases the count and an invalid action throws — practice before testing MiniBank.

CODE

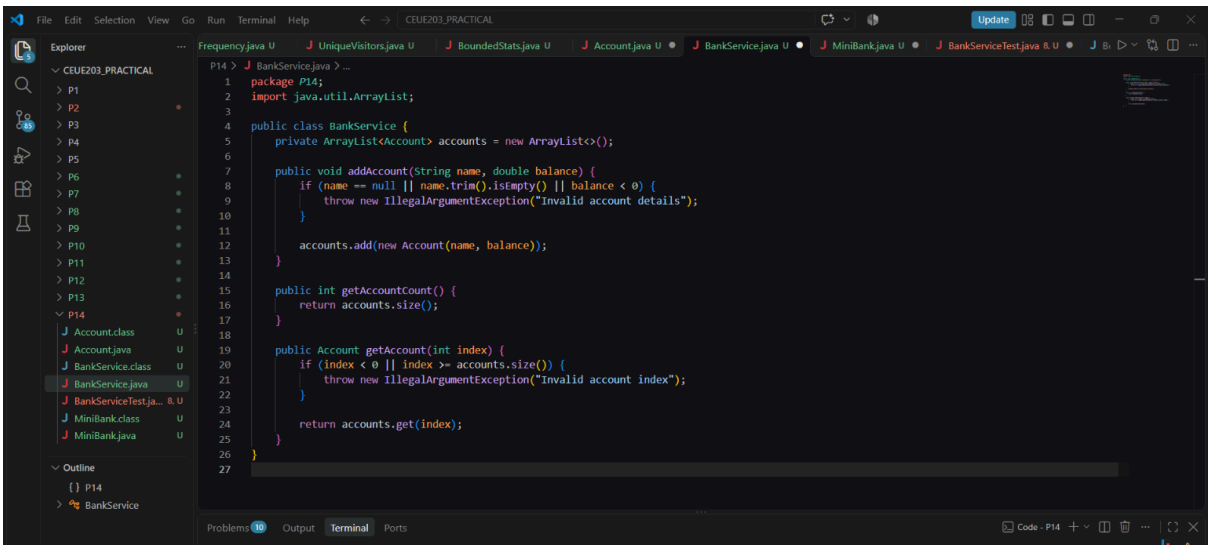


```java
package P14;
public class Account {
    private String name;
    private double balance;

    public Account(String name, double balance) {
        this.name = name;
        this.balance = balance;
    }

    public String getName() {
        return name;
    }

    public double getBalance() {
        return balance;
    }
}
```



```java
package P14;
import java.util.ArrayList;

public class BankService {
    private ArrayList<Account> accounts = new ArrayList<>();

    public void addAccount(String name, double balance) {
        if (name == null || name.trim().isEmpty() || balance < 0) {
            throw new IllegalArgumentException("Invalid account details");
        }

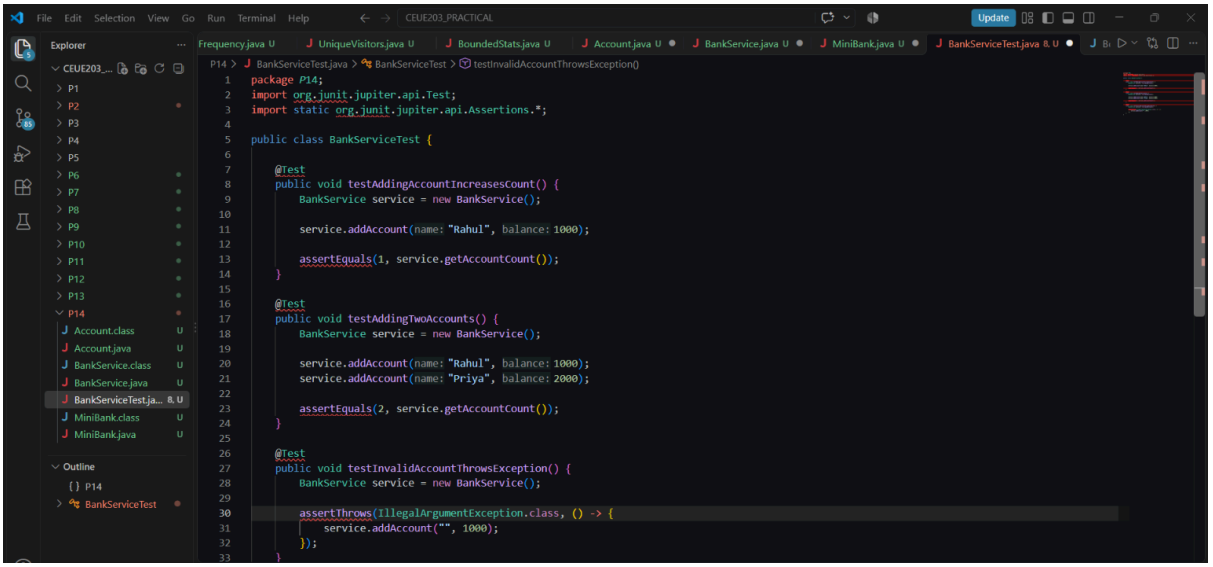
        accounts.add(new Account(name, balance));
    }

    public int getAccountCount() {
        return accounts.size();
    }

    public Account getAccount(int index) {
        if (index < 0 || index >= accounts.size()) {
            throw new IllegalArgumentException("Invalid account index");
        }

        return accounts.get(index);
    }
}
```

25IT036 CEUE203



```java
package P14;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class BankServiceTest {

    @Test
    public void testAddingAccountIncreasesCount() {
        BankService service = new BankService();

        service.addAccount(name: "Rahul", balance: 1000);

        assertEquals(1, service.getAccountCount());
    }

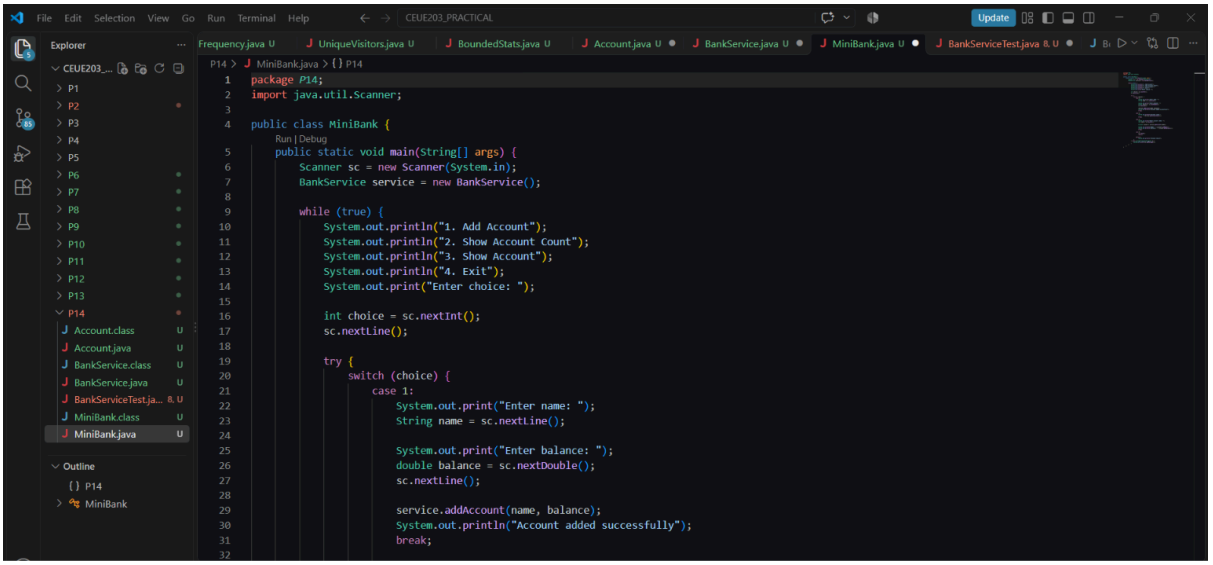
    @Test
    public void testAddingTwoAccounts() {
        BankService service = new BankService();

        service.addAccount(name: "Rahul", balance: 1000);
        service.addAccount(name: "Priya", balance: 2000);

        assertEquals(2, service.getAccountCount());
    }

    @Test
    public void testInvalidAccountThrowsException() {
        BankService service = new BankService();

        assertThrows(IllegalArgumentException.class, () -> {
            service.addAccount("", 1000);
        });
    }
```



```java
package P14;
import java.util.Scanner;

public class MiniBank {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankService service = new BankService();

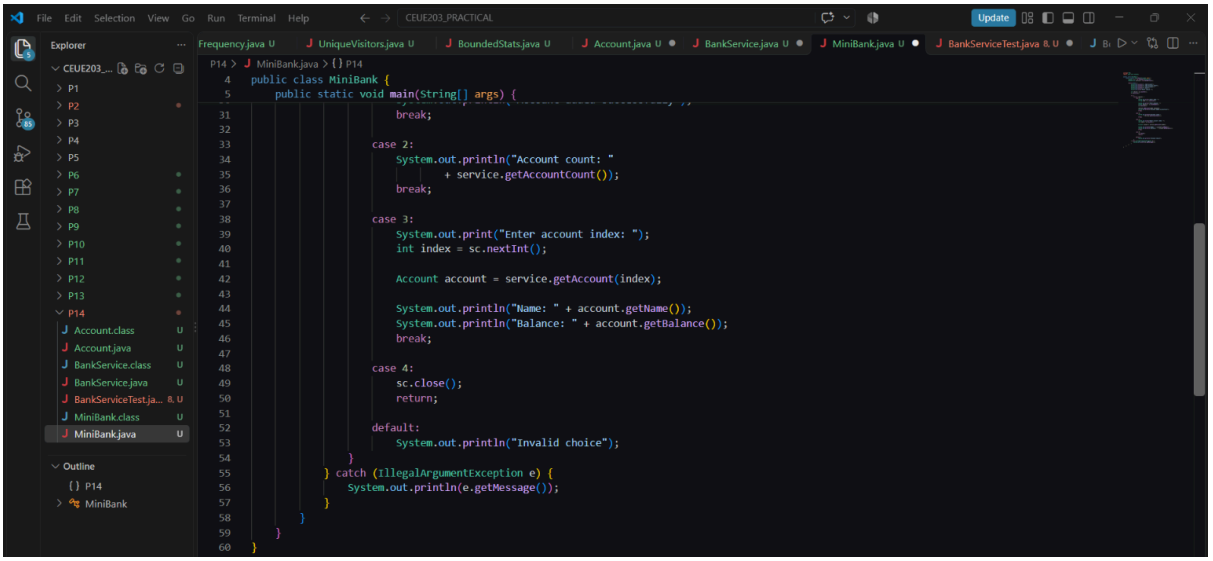
        while (true) {
            System.out.println("1. Add Account");
            System.out.println("2. Show Account Count");
            System.out.println("3. Show Account");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");

            int choice = sc.nextInt();
            sc.nextLine();

            try {
                switch (choice) {
                    case 1:
                        System.out.print("Enter name: ");
                        String name = sc.nextLine();

                        System.out.print("Enter balance: ");
                        double balance = sc.nextDouble();
                        sc.nextLine();

                        service.addAccount(name, balance);
                        System.out.println("Account added successfully");
                        break;
```



```java
                        break;

                    case 2:
                        System.out.println("Account count: "
                                + service.getAccountCount());
                        break;

                    case 3:
                        System.out.print("Enter account index: ");
                        int index = sc.nextInt();

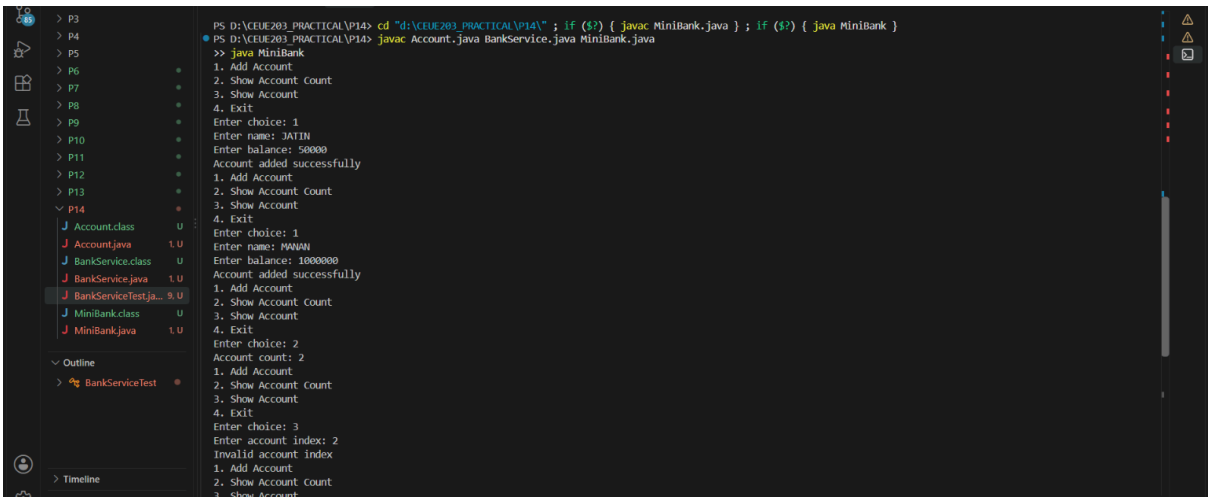
                        Account account = service.getAccount(index);

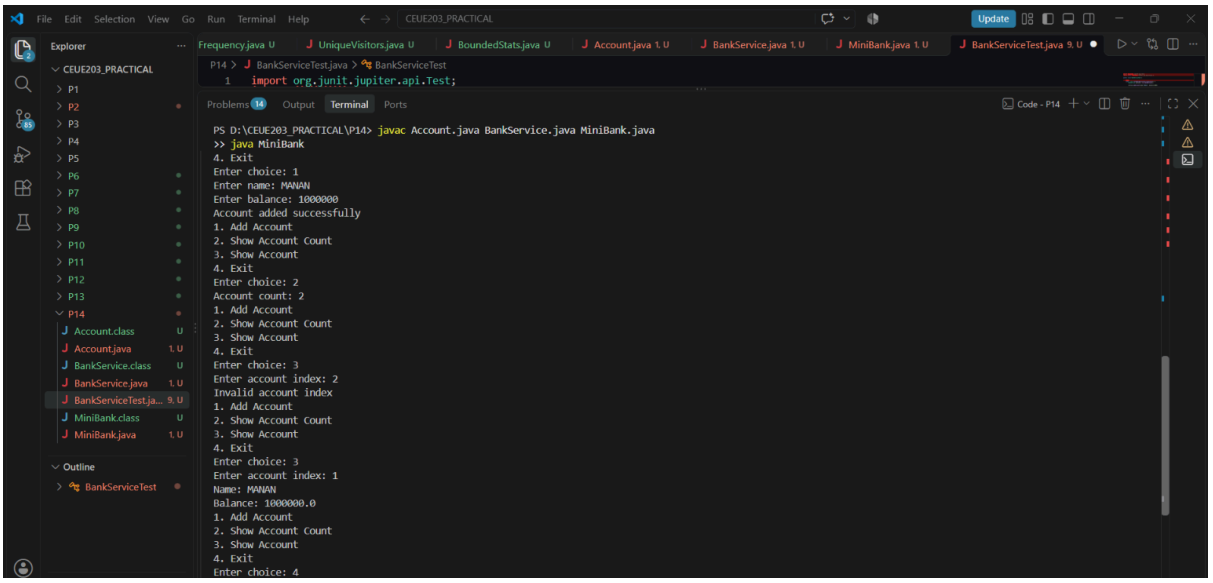
                        System.out.println("Name: " + account.getName());
                        System.out.println("Balance: " + account.getBalance());
                        break;

                    case 4:
                        sc.close();
                        return;

                    default:
                        System.out.println("Invalid choice");
                }
            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

OUTPUT




KEY QUESTIONS :

1) What is a layered architecture (model, service, user interface), and why is it useful?

Layered architecture divides an application into separate layers. The Model stores data, the Service contains business logic, and the User Interface interacts with the user. It makes the application easier to understand, test, maintain, and modify.

2) How do you assert in JUnit that a method throws an expected exception?

In JUnit 5, we use assertThrows() to check whether a method throws the expected exception.

Example:

```
assertThrows(IllegalArgumentException.class, () -> {
    service.addAccount("", 1000);
```

});

The test passes if the method throws IllegalArgumentException.

3) What does packaging the application as a runnable JAR provide?

A runnable JAR packages compiled Java classes and other required resources into a single file. It makes the application easier to distribute and run using the Java command, for example, java -jar MiniBank.jar.